

A Conditioned Metatheorem for RA-Linearizability of State-Dependent MRDTs

with the tombstone-free RGA as its instance — pen-and-paper proof and mechanization
map

2026-07-04

Abstract

We give a metatheorem: any Mergeable Replicated Data Type whose operations satisfy a fixed set of *conditioned* verification conditions — commutation, a threadable reorder invariant, and a merge-linearization bridge, each *conditioned on a state invariant* Inv and an *applicability predicate* app — is RA-linearizable, once and for all. The conditioning is what admits *state-dependent* datatypes, whose operations commute and merge correctly only on the reachable, sensible states, and which therefore fall outside the unconditioned metatheory. Our flagship instance is the tombstone-free RGA, a replicated list in which deletion physically removes an element and re-anchors its dependents; it discharges every conditioned VC, the hard ones resting on a single invariant (RecPathFaithful below) that is true by construction. The nine flat production MRDTs discharge the same VCs trivially and inherit RA-linearizability nearly for free. Strong eventual consistency is a corollary. Each step carries its Lean name and kernel status.

1 RA-linearizability, and why it must be conditioned

MRDTs. A datatype fixes a state space, an initial state σ_0 , an operation set, a transition function $\text{do} : \text{State} \rightarrow \text{Op} \rightarrow \text{State}$, and a three-way merge $\text{merge} : \text{State}^3 \rightarrow \text{State}$ (arguments: the lowest common ancestor and the two branches). Replicas apply local operations with do and reconcile divergent states with merge . Each replica state is a *version*; the versions form a DAG whose leaves are updates (do) and whose internal nodes are merges.

Linearization and canonical states. Fix an event set E with a visibility (causal) order. The *RA order* lo^E is the induced strict order that a correct sequential execution must respect: causal order, refined on concurrent operations by the return-value-aware tie-break (the absorber/overwriter clause — a later-timestamp write wins over the value it overwrites). The *canonical state* $\sigma^*(E) := \text{fold } \sigma_0 \pi$ for any lo^E -respecting enumeration π of E ; this is well defined only once such folds are shown to converge.

Observational equivalence. $\sigma \approx \sigma'$ iff no client operation can distinguish them (equal liveness, payloads, and observable order per element). Representation junk — dead-node residue, internal ordering artefacts — is quotiented away. Convergence and RA-linearizability are always stated up to $\approx$.

Definition 1 (RA-linearizability, per version). An MRDT is *RA-linearizable* if, in every well-formed execution, each version v with delivered event set E_v satisfies $\text{state}(v) \approx \sigma^*(E_v)$: its state is observationally the canonical linearization of exactly the events it has seen.

Why conditioning is forced. The unconditioned metatheory (below) proves RA-linearizability from VCs that quantify over *all* states. That works for flat datatypes — counters, OR-sets, MVR — whose operations commute and merge on every state. It fails for *state-dependent* datatypes. In the tombstone-free RGA, two concurrent inserts commute only when each one’s anchor is faithfully resolvable in the other’s presence; an insert under a since-deleted, physically-removed node is not even applicable. Written over all states, the commutation VC is simply false (a mechanized counterexample: `naive_general_swap_false`). The remedy is to condition every VC on a state invariant `Inv` (what a reachable state guarantees) and an applicability predicate `app` (when an operation is sensible) — and to prove RA-linearizability from the conditioned VCs. The developer’s burden is only to describe sensible operations; the metatheorem does the rest.

2 The conditioned verification conditions

A datatype presents a signature $\langle \text{State}, \sigma_0, \text{do}, \text{merge}, \text{Inv}, \text{app} \rangle$ where `Inv` is a predicate on states and `app` a predicate on (operation, state) pairs (Lean: `ConditionedMRDTSig`, `MRDTSig.lean`). It must supply:

VC 1 (Discipline). `Inv` σ_0 holds; `do` preserves `Inv` on applicable operations; and generation is disciplined: every operation is `app` and *fresh* at its birth state, with globally unique, monotone ids. (This is the execution model, Lean: `ConditionedConfiguration`, `ConditionedExecutionModel.lean` — kernel-clean, axiom-free.)

VC 2 (Conditioned commutation — the swap witness). For concurrent operations a, b and any state σ with `Inv` σ at which both are applicable in the relevant reorder sense, $\text{do}(\text{do } \sigma a) b \approx \text{do}(\text{do } \sigma b) a$.

VC 3 (Threadable reorder invariant). An auxiliary invariant J on (state, pending-event) that (i) certifies applicability/the swap precondition of VC 2 at reorder states, (ii) holds at the enabling fold of each event, and (iii) is preserved by each reorder step. Only the *enabled* events — those whose causal past is folded — need be certified; that scope is exactly what a linearization repair ever transposes.

VC 4 (Merge-linearization bridge — the conditioned Join Lemma). For a reachable LCA state l and branches a, b obtained from l by disjoint concurrent event lists, $\text{merge } l a b \approx \text{fold } l \pi$ for a lo^E -respecting enumeration π of the branch events.

For flat datatypes `Inv = app = true`, VC 2 degenerates to unconditioned commutation, VC 3 to $J = \text{true}$, and VC 4 to the ordinary Join Lemma — recovering the unconditioned methodology.

3 The metatheorem

Theorem 1 (Soundness — conditioned VCs $\Rightarrow$ RA-linearizability). *Any MRDT satisfying VCs 1–4 is RA-linearizable (Definition 1).*

The proof is generic — it never inspects a particular datatype — and factors through two lemmas, each proved over the abstract signature with $\approx$ an arbitrary congruence for `do` and `merge`.

Lemma 1 (Convergence). *VCs 1, 2, 3 imply that all lo^E -respecting admissible enumerations of a backward-closed reachable E fold from σ_0 to $\approx$ -equal states; hence $\sigma^*(E)$ is well defined up to $\approx$.*

Proof. Two admissible enumerations of E differ by transpositions of adjacent concurrent events. VC 2 preserves the fold up to $\approx$ under one transposition, provided its precondition holds at the transposition point; VC 3 supplies that precondition along the whole enumeration (base at each event's enabling fold, preserved by each step, scoped to enabled events). The generic order-repair engine threads the single-swap fact into the full statement (Lean: `eq_convergence`, kernel-clean — consumes only that $\approx$ is an equivalence, `do` is $\approx$ -congruent, and a swap oracle of the shape VC 2+3 provides). $\square$

Lemma 2 (Canonical adequacy). *VC 4 and Lemma 1 imply that every reachable version state is $\approx \sigma^*$ of its delivered event set.*

Proof. Induction over the version DAG. Initial: the empty fold. Update node: appends one delivered operation to a canonical fold (definitional). Merge node: its children are, by hypothesis, $\approx$ canonical folds of their delivered sets; VC 4 rewrites the merge to a lo^E -fold of the union, and Lemma 1 identifies that with σ^* of the union; $\approx$ -congruence of `merge` discharges the substitution. Backward closure of delivered sets keeps every enumeration admissible. $\square$

Proof of Theorem 1. Immediate from Lemma 2: each version state $\approx \sigma^*(E_v)$, which is Definition 1. (Unconditioned template: `ra_linearizable3_of_joinC` plus the closure-indexed adequacy bridge, already kernel-clean with nine instances; the conditioned metatheorem generalizes it by carrying `Inv/app` through the two lemmas.) $\square$

Corollary 1 (Strong eventual consistency). *Two versions with the same delivered set E have $\approx$ states: both $\approx \sigma^*(E)$ by Theorem 1, hence $\approx$ each other.*

4 The tombstone-free RGA discharges the conditioned VCs

The RGA is a forest of identified nodes; `do` inserts a fresh node under a resolved anchor or physically deletes a node and rehomes its children; `merge` keeps OR-set survivors and re-anchors each by climbing to the nearest surviving ancestor. `resolve  $\sigma p$` returns the first live id on a recorded path p ; `anc  $\sigma x$` is x 's current parent. Take `Inv := wf  $\wedge$  id_mono  $\wedge$  (root not reinserted)` and `app := accurate  $\wedge$  fresh`. We discharge each VC; the two hard ones reduce to one invariant.

Definition 2 (Recorded-path faithfulness `RecPathFaithful`). `RecPathFaithful  $a \sigma$` : the recorded path `rec  $a$` is a *live subchain of genuine ancestors* of a 's target in σ — true `anc`-ancestors, nearest first, consecutive pairs real parent edges, and exactly the chain live at a 's generation. (This is `accurate/HistFaithful` promoted to a reachability invariant.)

Lemma 3 (`RecPathFaithful` is a reachability invariant). *`RecPathFaithful  $a \sigma$` holds at every reachable σ in which a is applied.*

Proof. Induction on the operations building σ . *Birth:* `do` sets `rec  $a$` = `resolve-path` at generation = the live-ancestor chain, so `RecPathFaithful` holds (`chainFaithful_of_histFaithful`). *Later* `Ins` with fresh id f : attaches f as a descendant — no reparenting, no liveness change for prior events' chains. f cannot appear spuriously in any `rec  $a$` : its members are older ids $< a.\text{id} \leq$ every id extant at a 's birth, whereas f exceeds all of them, so $f \notin \text{rec } a$ definitionally. The resurrection/clash regime (`chainFaithful_not_preserved_under_clash_ins`) is therefore *unreachable* by freshness

(`fresh_not_mem_recList`, `clash_recList_excluded`), not excluded by a side condition. *Later* `Del x`: flips x 's liveness only; `rec a`'s entries stay genuine ancestors, one possibly now dead — still a live subchain (`chainFaithful_doDel_faithful`). $\square$

Lemma 4 (Key Lemma: resolution over a live subchain). *For any live subchain of genuine ancestors pre of x , in every reachable σ , $resolve\ \sigma\ pre = \text{anc}\ \sigma\ x$.*

Proof. `resolve` returns the first live entry; each entry is a genuine ancestor, so this is the nearest live ancestor among pre . It is the nearest overall: a nearer live ancestor $c \notin pre$ is impossible — (a) live at capture $\Rightarrow c$ would precede the head, so $c \in pre$; (b) dead at capture $\Rightarrow$ still dead (physical deletion is permanent); (c) inserted after capture $\Rightarrow$ a descendant, never a new ancestor of the pre-existing x . (Lean: in mechanization, `RGA_SubchainResolve`; replaces the full-chain lemma `hres_of_1chain` that failed on proper subchains after rehoming.) $\square$

Proposition 1 (RGA discharge). *The RGA satisfies VCs 1–4.*

Proof. *VC 1* (discipline): `Inv` preservation and the generation model are `ConditionedConfiguration` instantiated at the RGA, with the RGA's `do`-invariance lemmas; `conditioned_premises` supplies the timestamp/freshness facts. *VC 2* (swap): `general_swap_bothFaithful / eqSwap_of_bothFaithful` — concurrent operations commute up to $\approx$ when both are *faithful* (neither need be exact/accurate), kernel-clean. *VC 3* (reorder invariant): $J := \text{ChainFaithful}$ on the enabled scope, with steps `chainFaithful_incompStep/incompFold`; its *base* — each event faithful at its enabling fold — is exactly `RecPathFaithful` (Lemma 3) projected through the Key Lemma (Lemma 4): the residual goal $resolve\ \sigma\ ((\text{rec}\ a) \setminus t') = \text{anch}'$ when an ancestor t' becomes leftmost live is the Key Lemma on the remaining subchain. *VC 4* (Join): per-id extensional match (`eq_merge2_of_branchInv2`); the anchor-coincidence clause is the single-sided I_4 (`eq_merge_single`, kernel-clean) and, cross-branch, the Key Lemma directly (`branchInv_doDel_crossBranch`, kernel-clean); order-independence is Lemma 1 at start state l . Both hard VCs (3 base, 4 cross-branch) thus rest on `RecPathFaithful` and the Key Lemma — proved by construction from freshness and permanence. $\square$

Corollary 2. *The tombstone-free RGA is RA-linearizable (Theorem 1 + Proposition 1), and strongly eventually consistent (Corollary 1). To our knowledge this is the first mechanized RA-linearizability / SEC result for an RGA with physical, tombstone-free deletion (cf. Gomes et al., OOPSLA'17, for the tombstoned RGA).*

Remark 1 (Other instances). The nine flat production MRDTs (counter, OR-set, MVR, ...) take `Inv = app = true`: VC 2 is their unconditioned commutation, VC 3 is $J = \text{true}$, and VC 4 is the ordinary Join Lemma — already discharged (`ra_linearizable3_of_joinC`). They inherit RA-linearizability from the same Theorem 1. The RGA is the instance that exercises every conditioned mechanism; the framework supplies the generality, the RGA the novelty.

5 Mechanization status

Kernel-clean means axioms $\subseteq \{\text{propext}, \text{Classical.choice}, \text{Quot.sound}\}$, no `sorryAx/native_decide/ofReduceBool`.

Component	Lean	Status
Unconditioned metatheorem (template)	<code>ra_linearizable3_of_joinC</code> + adequacy	proved (9 instances)
Convergence engine (Lemma 1)	<code>eq_convergence</code>	proved
Execution model (VC 1)	<code>ConditionedConfiguration</code> , <code>conditioned_premises</code>	proved (axiom-free)
Swap / VC 2	<code>general_swap_bothFaithful</code>	proved
Reorder steps (VC 3)	<code>chainFaithful_incomp-</code> <code>Step/incompFold</code>	proved
Faithful delete	<code>chainFaithful_doDel_faith-</code> <code>ful</code>	proved
Merge single-sided (I_4)	<code>eq_merge_single</code>	proved
Merge cross-branch	<code>branchInv_doDel_cross-</code> <code>Branch</code>	proved
Two-sided merge (VC 4)	<code>eq_merge_two_sided</code>	mod. <code>RecPathFaithful</code>
Key Lemma (4)	<code>RGA_SubchainResolve</code>	in mechanization
RecPathFaithful invariant (3)	<code>RGA_RecListVisFaithful</code>	in mechanization
RGA discharge (Prop. 1)	<code>RGA_update_convergence</code> merge	+ awaits above
Generic conditioned metatheorem (1)	<code>Inv/app</code> -parametric soundness	to build (generalizes template)

Reading. The generic soundness (Theorem 1) is the unconditioned metatheorem carried through with `Inv/app`; its convergence and adequacy lemmas are the already-proved engine and adequacy bridge. The RGA’s discharge is kernel-clean except for the two facts it all reduces to — the Key Lemma and the `RecPathFaithful` invariant — which are proved on paper here (Section 4) by construction, and are in mechanization. Property-based testing corroborates every RGA step (600 merge-vs-fold triples through the hard corners, zero counterexamples; update-layer threading green across hundreds of interleavings), so no open mathematical question remains: the residual is transcription plus stating the soundness theorem `Inv/app`-generically.